Chatbot 請求路徑圖解

IP、Port、前端如何打到後端、Middleware 在做什麼 · 30 天 K8s + CI/CD 專案 · Day 12 補充教材

這份文件回答 4 個問題

1. **IP 和 Port 到底各有幾種?** 為什麼 Pod 的 IP 會一直變、Service 卻不會?
2. **我按下「送出」之後,請求走過哪些地方?** 每一站用的是哪個 port?
3. **前端的 JavaScript 怎麼知道要打哪個後端?** 為什麼會有「預檢請求」(preflight)?
4. **Middleware 是什麼?** 在你的專案裡它做了哪兩件事、順序為什麼重要?

圖中的顏色代表什麼

瀏覽器、外部　K8s 網路層(IP 穩定)　Pod(IP 會變)　重點

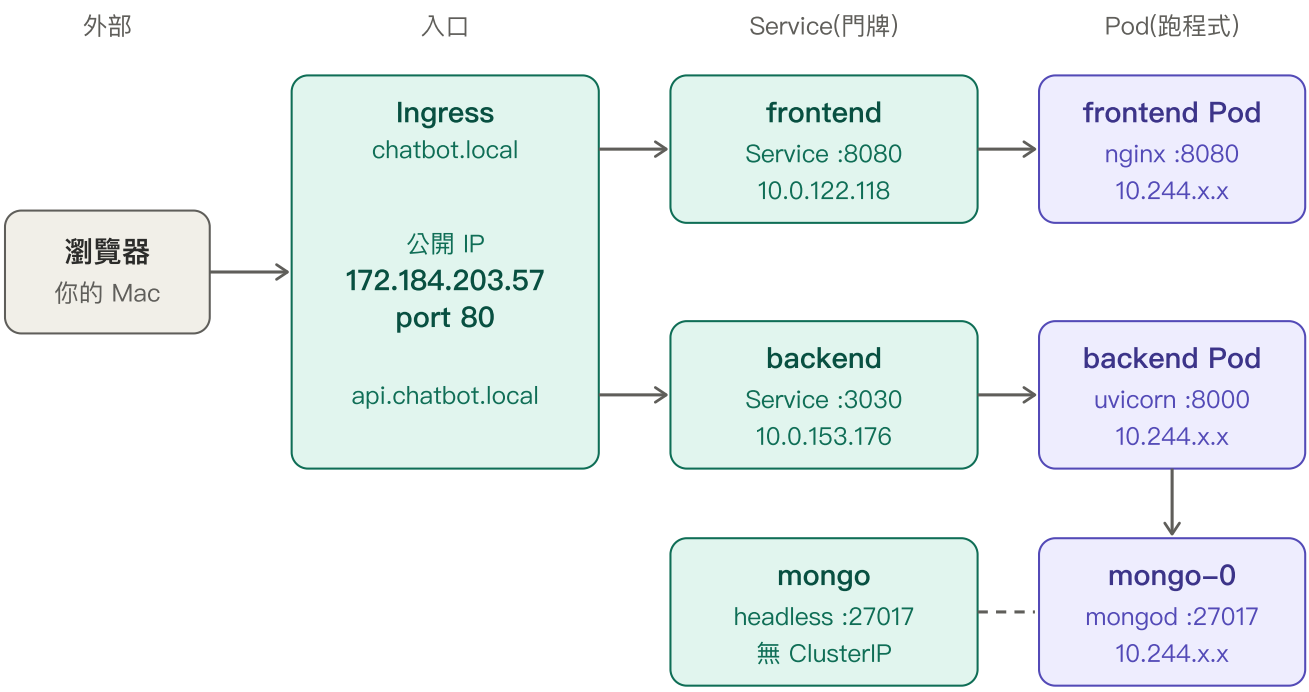
> **先記住一句話:**IP 回答「去哪棟大樓」,Port 回答「找哪一個櫃台」。Pod 這棟大樓每天搬家,所以前面放一個不搬家的 Service 當總機。

本文件的實測依據

文中的數字來自你專案的實際檔案與叢集:`mychart/values.yaml`、`chatbot-frontend/src/api.js`、`chatbot-backend/main.py`,以及 Day 12 驗證時查到的 Ingress 公開 IP 與 Service ClusterIP。

圖中的 `10.244.x.x` 與 `10.0.x.x` 是範例:Pod IP 每次重啟都會變,實際數字請以 `kubectl get pods -o wide` 為準。

一、全景圖:請求經過的每一站



虛線:mongo 的 Service 只負責「名字解析」,DNS 直接指到 mongo-0 的 Pod IP。

三種 IP

開頭	是什麼	穩定嗎	誰會用
`172.184.x.x`	公開 IP:Azure 給 Ingress 的對外大門	穩定	你的瀏覽器
`10.0.x.x`	Service 的 ClusterIP:叢集內的固定門牌	穩定	叢集內其他元件
`10.244.x.x`	Pod 的 IP:真正跑程式的那個	**每次重啟都換**	幾乎沒人直接用

三種 Port

port	意思	在圖上
Ingress 的 `80`	大門的門號	瀏覽器打這個
Service 的 port(`3030` 、 `8080`)	總機的「分機號碼」,**你自己挑的**	Service 方塊裡 `:3030`
Pod 的 port(`8000` 、 `8080` 、 `27017`)	程式**真正**在聽的門,由程式決定	Pod 方塊裡 `:8000`

> **backend 就是:**總機 `3030` ——轉接(targetPort)——→ 程式 `8000` 。3030 是你自己選的,8000 是 uvicorn 寫死的,兩者可以不同。

二、一次完整旅程:你按下「送出」

1. **查網域。**瀏覽器要連 `api.chatbot.local`,查你電腦的 `/etc/hosts`(Day 6 改的),得到 `172.184.203.57`。
2. **敲大門。**連到該 IP 的 **port 80**,也就是 Ingress。
3. **看網域分流。**Ingress 看請求裡的網域:是 `api.chatbot.local` → 轉給 backend Service。
4. **總機轉接。**backend Service(`:3030`)從背後的 Pod 中挑一個,轉進它的 `:8000`。
5. **處理請求。**uvicorn 收到後執行 `/chat`:呼叫 Gemini,把這一輪對話寫進 Mongo。
6. **連資料庫。**backend Pod 用名字 `mongo` 查 DNS,得到 mongo-0 的 Pod IP,連它的 `:27017`。
7. **回程。**回應沿原路返回給瀏覽器。

> **兩個容易忽略的重點**
> ① 分流的依據是**網域名稱**,不是 IP:`chatbot.local` 與 `api.chatbot.local` 其實是**同一個 IP、同一個 port 80**,Ingress 靠網域才知道要送哪邊。
> ② **frontend Pod 從頭到尾沒有打過 backend。**它只負責把網頁檔案(nginx)交給瀏覽器。聊天時是**瀏覽器裡的 JavaScript** 自己去打 `api.chatbot.local`,所以全景圖裡兩條路是平行的,不是串聯。

其他你會遇到的 port(不在全景圖裡)

你會看到	是什麼
Prometheus 抓 `Pod IP:8000/metrics`	**直接連 Pod**,不經過 Service 的 3030(Day 12 的 target 就是這樣)
`localhost:3001`(Grafana)	`kubectl port-forward` 把你 Mac 的 3001 接到 Grafana 的 80,只在你電腦上有效
`localhost:9090`、`18030`	同理,port-forward 的臨時通道。數字是自己隨便挑的,關掉就沒了

為什麼要有 Service?

Pod 會因為重啟、更新、HPA 擴縮而被換掉,**每次 IP 都不同**。如果 Ingress 直接記 Pod IP,Pod 一換就找不到人。Service 提供一個**固定的 IP 與名字**,並自動追蹤背後有哪些健康的 Pod,把流量分給它們。這也是 HPA 擴到 5 個 Pod 時,呼叫端完全不用改任何設定的原因。

你專案裡所有數字對照表

數字	屬於	誰決定
`80`	Ingress 對外門號	ingress-nginx 預設
`3030`	backend Service	**你**(Day 4)
`8080`	frontend Service 與 nginx	**你**(`nginx.conf`)
`8000`	backend Pod(uvicorn)	程式(Dockerfile / main.py)
`27017`	Mongo	MongoDB 預設

三、前端 JavaScript 怎麼打到後端

關鍵觀念:網頁載入後,**JavaScript 是在你的瀏覽器裡執行**,不是在 frontend Pod 裡。所以「打後端」這個動作,是你的瀏覽器發出的。

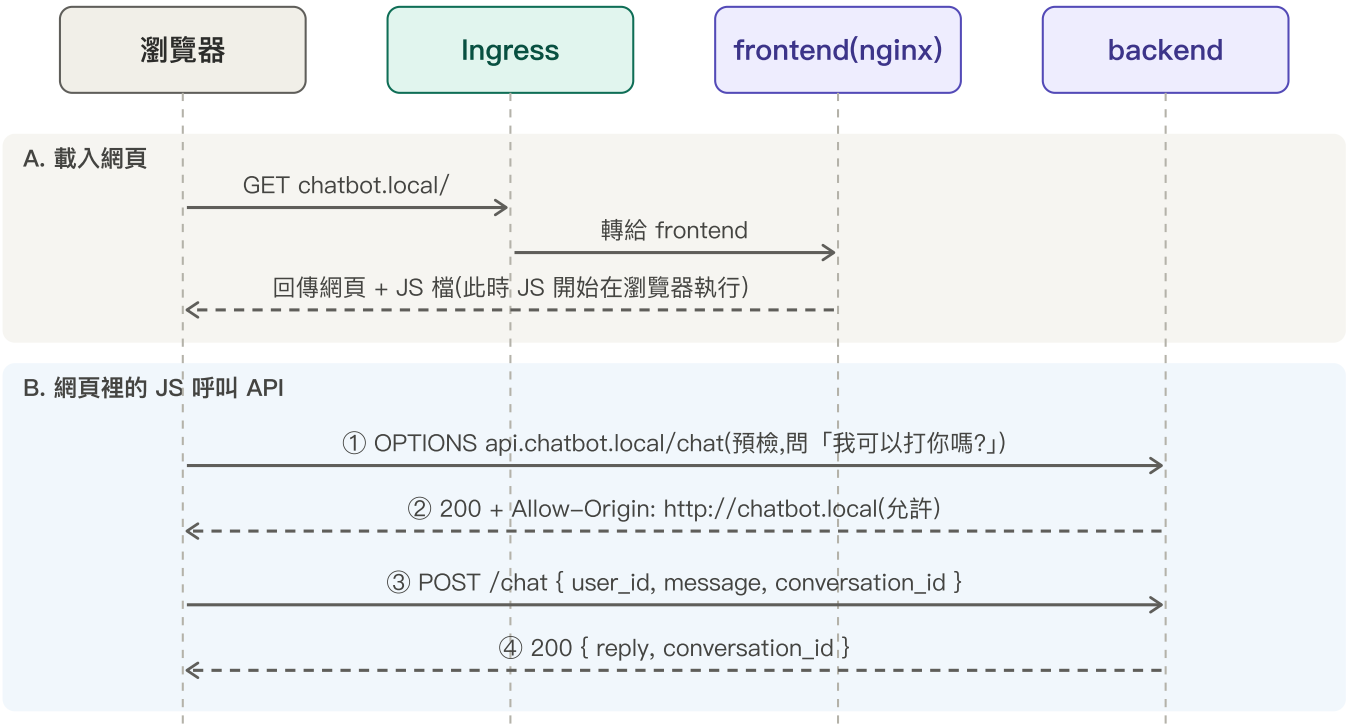
後端網址從哪來?

```
// chatbot-frontend/src/api.js
const API_BASE_URL = process.env.REACT_APP_API_URL || "http://api.chatbot.local";
...
fetch(`${API_BASE_URL}/chat`, { method: "POST", ... })
```

你的專案沒有設定 `REACT_APP_API_URL`,所以用預設值 `http://api.chatbot.local`。這個值在 `npm run build` 時就被**烤進靜態檔案**,不是容器啟動時才讀。

> **所以:**想換後端網址,必須**重新 build frontend image**(改 ConfigMap 或重啟 Pod 都沒用)。這是 CRA(`REACT_APP_` 開頭的變數)的規則。

時序圖:載入網頁,然後呼叫 API



B 區的箭頭都會先經過 Ingress,再轉給 backend;圖中為了簡潔直接畫到 backend。

為什麼會有 ① 預檢?CORS 是什麼?

瀏覽器有個安全規則(**同源政策**):網頁預設只能打「同一個來源」。來源 = 協定 + 網域 + port。你的網頁來自 `chatbot.local`,卻要打 `api.chatbot.local`,**網域不同 → 不同來源**(即使 IP 相同)。

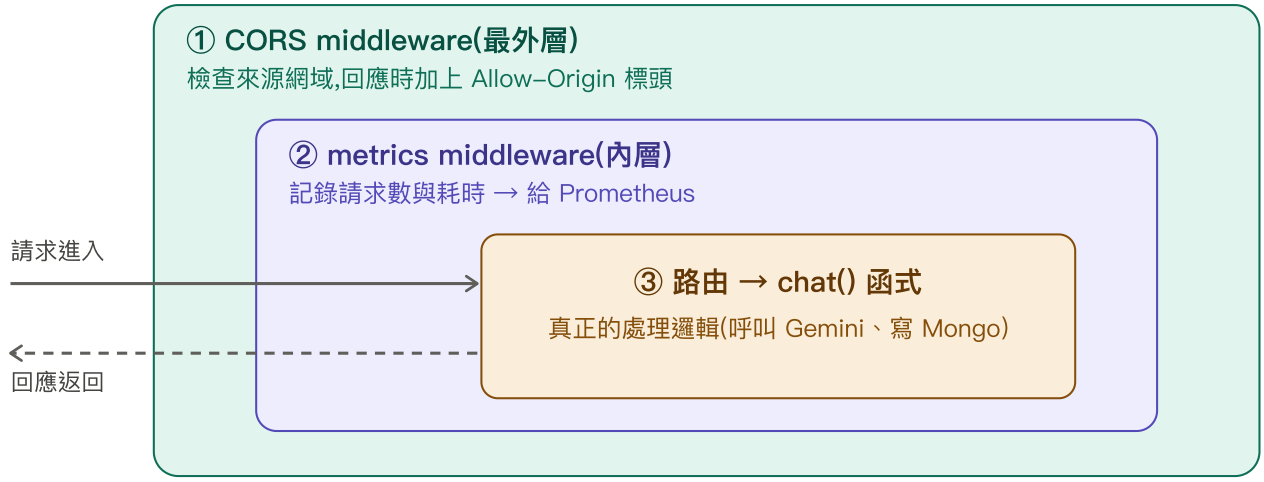
要放行,後端必須明確回應「我允許這個來源」,這套機制叫 **CORS**。又因為你送的是 JSON(`Content-Type: application/json`),瀏覽器會**先自動發一個 OPTIONS 預檢**探路,通過了才送真正的 POST。

四、Middleware 是什麼

Middleware(中介層)是夾在「請求進來」與「你的函式 `chat()`」之間的程式,**每個請求都會經過它**,回應返回時也會再經過一次。

比喻:像機場安檢。每位旅客(請求)登機前都要過安檢,下機時也要過海關。**不需要在每個登機門(每個 endpoint)各設一套,統一在入口處理。**

你的專案有兩層 middleware



層	請求進來時	回應返回時
① CORS	看 `Origin` 在不在白名單;是預檢就**直接回覆**,不往內傳	替回應加上 `Access-Control-Allow-Origin`
② metrics	開始計時	計算耗時,記錄 `path` / `status` / 延遲
③ chat()	跑業務邏輯	回傳結果

順序為什麼重要?(我實測過)

```
app.middleware("http")(metrics_middleware)   # 先加入 → 在內層
app.add_middleware(CORSMiddleware, ...)      # 後加入 → 在外層
```

規則:後加入的在外層。實測 `app.user_middleware` 順序為 `CORSMiddleware → BaseHTTPMiddleware`,與上圖一致。

實測	結果
預檢 OPTIONS,來源 `http://chatbot.local`(正確)	200,回 `Allow-Origin: http://chatbot.local`
預檢 OPTIONS,來源 `http://evil.example`(錯誤)	**400 Disallowed CORS origin**,沒有 Allow-Origin → 瀏覽器會擋下,真正的 POST 根本不會送出
預檢請求有沒有被 metrics 記到?	**沒有。**CORS 在最外層、直接回覆,請求到不了內層的 metrics

> **意義:**Grafana 上看到的 `/chat` 請求數,只算「真正的 POST」,不含瀏覽器自動發的預檢,數字比較乾淨。

五、面試考點

Q1. Ingress、Service、Pod 差在哪?
Pod 是真正跑程式的單位,IP 會變;Service 提供固定 IP/名字並做負載平衡,屏蔽 Pod 的更替;Ingress 是 HTTP 層的入口,依網域或路徑把外部流量分給不同 Service。

Q2. Service 的 `port` 與 `targetPort` 差在哪?
`port` 是 Service 對外(叢集內)開的號碼;`targetPort` 是轉進 Pod 的哪個 port。你的 backend 是 3030 → 8000,故意不同,才分得清楚。

Q3. headless Service 是什麼?為什麼 Mongo 用它?
`clusterIP: None`,沒有虛擬 IP,DNS 直接回傳 Pod IP。StatefulSet 需要穩定的個別 Pod 身分(`mongo-0`),不能被負載平衡到別的 Pod。

Q4. CORS 由誰執行?curl 會被擋嗎?
由**瀏覽器**執行,不是伺服器。後端只是回應標頭,真正擋下請求的是瀏覽器。所以 curl、Postman 都不受影響,這也是「curl 通、瀏覽器不通」的常見原因。

Q5. 為什麼會有 preflight?
跨來源且屬於「非簡單請求」(例如 JSON 的 Content-Type)時,瀏覽器會先發 OPTIONS 詢問伺服器是否允許;允許才送真正的請求。

Q6. Middleware 是什麼?順序有影響嗎?
包在每個請求外的共用處理層,用來做驗證、日誌、指標、CORS 等橫切關注點。有影響:後加入的在外層,外層先處理請求、最後處理回應。你的專案 CORS 在 metrics 外,所以預檢不會被計數。

Q7. 為什麼改了 `REACT_APP_API_URL` 要重新 build?
CRA 在 build 時把它替換成靜態字串寫進 JS 檔,執行時不再讀環境變數。若要執行期可換,可改用同源反向代理(`/api`),或在執行時載入設定檔。

常見踩雷與排查

症狀	先檢查
瀏覽器 Console 出現 `blocked by CORS policy`	ConfigMap 的 `CORS_ORIGINS` 是否**完全等於**頁面網址(含 `http://`、不含結尾斜線、port 要一致)
curl 通,瀏覽器不通	幾乎一定是 CORS(瀏覽器才擋)
`api.chatbot.local` 完全連不上	`/etc/hosts` 是否指到目前的 Ingress 公開 IP;cluster 是否已啟動
改了後端網址沒反應	前端網址是 build 時烤進去的,要重 build frontend image
Grafana 看不到 backend 指標	Service 的 `app: chatbot-backend` 標籤與 port 名字 `http` 是否與 ServiceMonitor 一致

自我檢查

1. 為什麼 `chatbot.local` 和 `api.chatbot.local` 是同一個 IP,卻算「不同來源」?
2. frontend Pod 有沒有呼叫過 backend?那是誰在呼叫?
3. Prometheus 抓指標時,用的是 3030 還是 8000?為什麼?
4. 如果把 CORS 與 metrics 的加入順序對調,預檢請求會不會被計數?
5. mongo 的 Service 為什麼沒有 ClusterIP?